Temat projektu:

System Ewidencji i Zarządzania Placówką Pocztową.

Wszelkie dane osobowe oraz adresowe wykorzystane w projekcie są fikcyjne i zostały stworzone wyłącznie na potrzeby celów edukacyjnych (symulacji działania systemu).

Główny cel:

Celem projektu jest stworzenie relacyjnej bazy danych, która usprawni codzienne funkcjonowanie urzędu pocztowego. (System ma zastąpić tradycyjne, papierowe rejestry, umożliwiając szybkie wyszukiwanie przesyłek oraz efektywne zarządzanie pracą listonoszy w przydzielonych im rejonach.)

Zastosowanie biznesowe (Funkcjonalności):

System realizuje kluczowe procesy biznesowe, wynikające bezpośrednio ze struktury bazy danych:

1. Ewidencja i śledzenie przesyłek:

Rejestracja każdej przesyłki (listy) wpływającej do placówki (numer nadawczy, adresat, data wpływu).

Monitorowanie statusu doręczenia (np. "Doręczono", "Awizo", "Zwrot"), co pozwala na szybką obsługę zapytań klientów.

2. Zarządzanie personelem (Listonosze):

Przechowywanie danych o zatrudnionych listonoszach (imię, nazwisko, nr legitymacji).

Monitorowanie obciążenia pracą poszczególnych pracowników.

3. Logistyka i Rejonizacja:

Przypisanie listonoszy do konkretnych rejonów doręczeń i punktów odbioru, co optymalizuje logistykę dostarczania poczty.

4. Kategoryzacja i Wycena usług:

Rozróżnienie typów przesyłek (np. czy wymaga podpisu), co jest kluczowe dla procedur prawnych.

Automatyzacja wyceny usług dzięki centralnemu cennikowi, który wiąże rodzaj listu i jego gabaryt z ceną końcową.

PUNKT 2. Projekt bazy danych

2.1. Zidentyfikowane encje i atrybuty

Baza danych składa się z czterech głównych encji, które odwzorowują rzeczywiste obiekty systemu pocztowego:

1. `rejony` (Słownik obszarów doręczeń)
* `id_rejonu` (PK) – Unikalny identyfikator rejonu.
* `nazwa_rejonu` – Nazwa dzielnicy lub obszaru (np. "Centrum").
* `punkt_odbioru` – Przypisana placówka pocztowa obsługująca dany rejon.

2. `rodzaje_listow` (Słownik typów przesyłek)
* `id_rodzaju` (PK) – Unikalny identyfikator typu.
* `nazwa_rodzaju` – Nazwa kategorii (np. "Polecony", "Zwykły").
* `wymaga_podpisu` – Flaga logiczna (Tak/Nie) określająca procedurę odbioru.

3. `listonosze` (Pracownicy)
* `id_listonosza` (PK) – Unikalny identyfikator pracownika.
* `imie`, `nazwisko` – Dane osobowe.
* `nr_legitymacji` – Unikalny numer służbowy.
* `id_rejonu` (FK) – Przypisanie pracownika do obsługiwanego rejonu.

4. `ewidencja_listow` (Główny rejestr operacyjny)
* `id_listu` (PK) – Identyfikator systemowy przesyłki.
* `numer_nadawczy` – Unikalny numer śledzenia przesyłki.
* `adresat_nazwa`, `adresat_adres` – Dane odbiorcy.
* `data_wplywu` – Data przyjęcia przesyłki do placówki.
* `status_doreczenia` – Aktualny stan (np. "W doręczeniu", "Awizo").
* `id_rodzaju` (FK) – Kategoria przesyłki.
* `id_listonosza` (FK) – Pracownik odpowiedzialny za doręczenie.

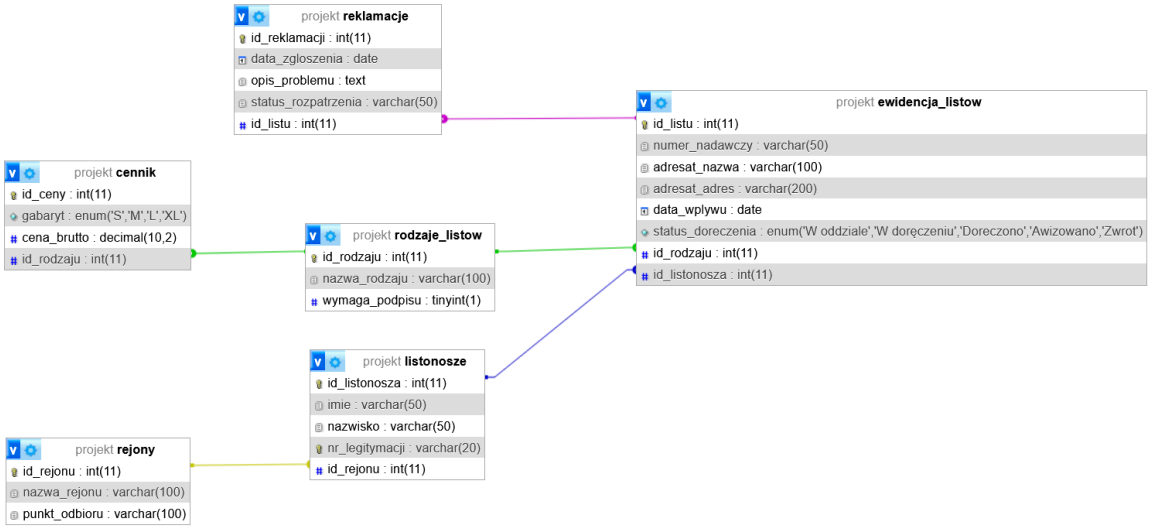
5.’cennik’ (Finanse)
* `id_ceny` (PK) – Identyfikator pozycji w cenniku.
* `gabaryt` – Oznaczenie wielkości przesyłki (S/M/L/XL).
* `cenna_brutto` – Koszt realizacji usługi pocztowej.
* `id_rodzaju` (FK) – Powiązanie ceny z konkretnym typem przesyłki.

2.2. Relacje i Normalizacja
Powtarzalne dane na przykład (nazwy rejonów, rodzaje listów) zostały przeniesione do osobnych tabel (`rejony`, `rodzaje_listow`), co eliminuje redundancję danych.

Relacje:

* Relacja 1:N (Jeden do Wielu) między `rejony` a `listonosze` (Jeden rejon może mieć wielu listonoszy).
* Relacja 1:N (Jeden do Wielu) między `listonosze` a `ewidencja_listow` (Jeden listonosz obsługuje wiele listów).
* Relacja 1:N (Jeden do Wielu) między `rodzaje_listow` a `ewidencja_listow` (Jeden rodzaj listu może wystąpić w wielu przesyłkach).
*Relacja 1:N (Jeden do Wielu) między rodzaje_listow a cennik (Jeden rodzaj listu, np. paczka, posiada wiele wariantów cenowych w zależności od gabarytu).
*Relacja 1:N (Jeden do Wielu) między ewidencja_listow a reklamacje (Do jednej przesyłki może zostać przypisanych wiele zgłoszeń reklamacyjnych).
*Relacja 1:N (Jeden do Wielu) między listonosze a pojazdy (Jeden listonosz może mieć przypisany do siebie pojazd służbowy, za który odpowiada).

2.3. Diagram (Entity-Relationship)



2.4. Implementacja (Kod SQL)
Poniżej znajduje się kod SQL (DDL) wykorzystany do utworzenia struktury bazy:

```sql
CREATE TABLE rejony (
    id_rejonu INT AUTO_INCREMENT PRIMARY KEY,
    nazwa_rejonu VARCHAR(100) NOT NULL,
    punkt_odbioru VARCHAR(200)
);

CREATE TABLE rodzaje_listow (
    id_rodzaju INT AUTO_INCREMENT PRIMARY KEY,
```

```
    nazwa_rodzaju VARCHAR(50) NOT NULL,
    wymaga_podpisu BOOLEAN DEFAULT FALSE
);

CREATE TABLE listonosze (
    id_listonosza INT AUTO_INCREMENT PRIMARY KEY,
    imie VARCHAR(50) NOT NULL,
    nazwisko VARCHAR(50) NOT NULL,
    nr_legitymacji VARCHAR(20) UNIQUE NOT NULL,
    id_rejonu INT,
    FOREIGN KEY (id_rejonu) REFERENCES rejony(id_rejonu)
);

CREATE TABLE ewidencja_listow (
    id_listu INT AUTO_INCREMENT PRIMARY KEY,
    numer_nadawczy VARCHAR(50) UNIQUE NOT NULL,
    adresat_nazwa VARCHAR(100) NOT NULL,
    adresat_adres VARCHAR(200) NOT NULL,
    data_wplywu DATE NOT NULL,
    status_doreczenia VARCHAR(20) DEFAULT 'W doręczeniu',
    id_rodzaju INT,
    id_listonosza INT,
    FOREIGN KEY (id_rodzaju) REFERENCES rodzaje_listow(id_rodzaju),
    FOREIGN KEY (id_listonosza) REFERENCES listonosze(id_listonosza)
);

CREATE TABLE cennik (
  id_ceny INT AUTO_INCREMENT PRIMARY KEY,
  gabaryt ENUM('S','M','L','XL') DEFAULT 'S',
  cena_brutto DECIMAL(10,2) NOT NULL,
  id_rodzaju INT NOT NULL,
  FOREIGN KEY (id_rodzaju) REFERENCES rodzaje_listow(id_rodzaju)
);
```

PUNKT 3. Scenariusz analitycznego wykorzystania

Poniżej przedstawiono zestaw instrukcji SQL realizujących kluczowe potrzeby informacyjne kadry zarządzającej urzędem pocztowym.

3.1. Raport efektywności pracowników

Cel biznesowy: Identyfikacja listonoszy obsługujących największą liczbę przesyłek w celu optymalizacji podziału pracy i premiowania najwydajniejszych pracowników.
Zastosowane funkcje: Złączenie tabel (`JOIN`), grupowanie (`GROUP BY`), funkcja agregująca (`COUNT`), sortowanie (`ORDER BY`).

sql

```sql
SELECT
    l.imie,
    l.nazwisko,
    COUNT(e.id_listu) AS liczba_przesylek
FROM listonosze l
JOIN ewidencja_listow e ON l.id_listonosza = e.id_listonosza
GROUP BY l.id_listonosza, l.imie, l.nazwisko
ORDER BY liczba_przesylek DESC;
```

imie	nazwisko	liczba_przesylek 1
Jan	Kowalski	4
Anna	Nowak	3
Marek	Wiśniewski	3
Katarzyna	Wójcik	3
Piotr	Zieliński	3
Tomasz	Nowicki	3
Monika	Szymańska	2
Michał	Kamiński	2
Barbara	Dąbrowska	2
Robert	Zieliński	2
Paweł	Woźniak	2
Agnieszka	Lewandowska	2
Krzysztof	Kozłowski	2

3.2 Raport największego ruchu

Cel biznesowy: Analiza natężenia ruchu pocztowego w poszczególnych dzielnicach miasta. Raport pozwala Kierownikowi zidentyfikować rejony o największej liczbie przesyłek, co jest podstawą do decyzji o ewentualnym przydzieleniu dodatkowych pojazdów lub wsparcia kadrowego w tych lokalizacjach.

```sql
SELECT
  r.nazwa_rejonu,
    COUNT(e.id_listu) AS liczba_przesylek
FROM rejony r
JOIN listonosze l ON r.id_rejonu = l.id_rejonu
JOIN ewidencja_listow e ON l.id_listonosza = e.id_listonosza
GROUP BY r.id_rejonu, r.nazwa_rejonu
ORDER BY liczba_przesylek DESC;
```

nazwa_rejonu	liczba_przesylek 1
Centrum	9
Jeżyce	5
Winogrady	5
Rataje	5
Grunwald	3
Piątkowo	2
Łazarz	2
Wilda	1

3.3. Analiza problemów z doręczeniami w rejonach

Cel biznesowy: Wykrycie obszarów (dzielnic), w których występuje najwięcej niedoręczonych przesyłek (awizowanych lub zwróconych), co może sugerować problemy z dostępem do skrzynek lub niewłaściwą pracę listonosza.
Zastosowane funkcje: Wielokrotne złączenie (`JOIN`), filtrowanie warunkowe (`WHERE`), grupowanie (`GROUP BY`).

```sql
SELECT
   r.nazwa_rejonu,
   COUNT(e.id_listu) AS niedoreczone_listy
FROM rejony r
JOIN listonosze l ON r.id_rejonu = l.id_rejonu
JOIN ewidencja_listow e ON l.id_listonosza = e.id_listonosza
WHERE e.status_doreczenia != 'Doręczono'
GROUP BY r.nazwa_rejonu
ORDER BY niedoreczone_listy DESC;
```

nazwa_rejonu	niedoreczone_listy 1
Centrum	9
Winogrady	5
Rataje	5
Jeżyce	5
Grunwald	3
Łazarz	2
Wilda	2
Piątkowo	2

3.4. Struktura realizowanych usług pocztowych

Cel biznesowy: Analiza zapotrzebowania na konkretne typy usług (np. listy polecone vs zwykłe) oraz weryfikacja, jaka część korespondencji wymaga dodatkowych procedur (podpis odbiorcy).
Zastosowane funkcje: Złączenie (`JOIN`), grupowanie (`GROUP BY`), filtrowanie kolumn.

sql:

```
SELECT
    rl.nazwa_rodzaju,
    rl.wymaga_podpisu,
    COUNT(e.id_listu) AS ilosc_przesylek
FROM rodzaje_listow rl
JOIN ewidencja_listow e ON rl.id_rodzaju = e.id_rodzaju
GROUP BY rl.nazwa_rodzaju, rl.wymaga_podpisu;
```

nazwa_rodzaju	wymaga_podpisu	ilosc_przesylek
Paczka MINI	0	3
Paczka Pocztowa	1	2
Polecony	1	5
Polecony z potw. odbioru	1	6
Priorytet	0	4
Przesyłka Pobraniowa	1	2
Sądowy	1	5
Zwykły	0	6

4. Scenariusz administracyjny

W tym rozdziale przedstawiono kluczowe modyfikacje struktury bazy danych, politykę bezpieczeństwa oraz przykłady codziennej pracy z danymi.

4.1. Modyfikacja struktury bazy

Wraz z rozwojem terytorialnym placówki pocztowej i zwiększeniem liczby listonoszy, pojawiła się konieczność ewidencjonowania środków transportu. Kierownik placówki zgłosił potrzebę przypisywania konkretnych pojazdów (rowerów, samochodów służbowych) do pracowników, aby monitorować stan techniczny floty i odpowiedzialność materialną.

Realizacja: Zaprojektowano i wdrożono nową tabelę `pojazdy`. Tabela ta przechowuje dane techniczne oraz posiada klucz obcy (`id_listonosza`), tworząc relację, która informuje, kto aktualnie użytkuje dany pojazd.

SQL

```
CREATE TABLE pojazdy (
  id_pojazdu INT AUTO_INCREMENT PRIMARY KEY,
  marka VARCHAR(50) NOT NULL,
  model VARCHAR(50) NOT NULL,
  nr_rejestracyjny VARCHAR(20),
  typ_pojazdu ENUM('Rower','Samochód','Skuter') NOT NULL,
  id_listonosza INT,
  FOREIGN KEY (id_listonosza) REFERENCES listonosze(id_listonosza)
```

);

4.2. Zarządzanie bezpieczeństwem

Potrzeba biznesowa: W celu ochrony danych osobowych klientów oraz zapobiegania celowemu usunięciu historii przesyłek.

Stworzymy trzy dedykowane role użytkowników o zróżnicowanych uprawnieniach:

1. Kierownik (Administrator): Pełny dostęp do zarządzania strukturą i danymi.
2. Pracownik (Obsługa klienta): Może dodawać i edytować listy, ale zablokowano mu możliwość usuwania danych (DELETE), co chroni bazę przed utratą historii.
3. Listonosz (Teren): Dostęp ograniczony wyłącznie do podglądu listów i aktualizacji ich statusu.

SQL

```
CREATE USER 'kierownik'@'localhost' IDENTIFIED BY 'admin';
GRANT ALL PRIVILEGES ON projekt.* TO 'kierownik'@'localhost';

CREATE USER ‘pracownik'@'localhost' IDENTIFIED BY 'pracownik';
GRANT SELECT, INSERT, UPDATE ON projekt.* TO 'pracownik'@'localhost';

CREATE USER 'listonosz'@'localhost' IDENTIFIED BY 'listonosz';
GRANT SELECT, UPDATE ON projekt.* TO 'listonosz'@'localhost';

FLUSH PRIVILEGES;
```

Tworzenie stażysty

```
CREATE USER 'stazysta'@'localhost' IDENTIFIED BY 'stazysta';
GRANT SELECT, INSERT, UPDATE ON projekt.* TO 'stazysta'@'localhost';
FLUSH PRIVILEGES;
```

4.3. Zarządzanie danymi

Poniżej przedstawiono przykładowe zapytania SQL realizujące codzienne operacje na danych.

A. Aktualizacja statusu przesyłki (UPDATE): Symulacja sytuacji, w której listonosz pobiera listy z oddziału i wychodzi w rejon. System zmienia status z "W oddziale" na "W doręczeniu".

SQL

```
UPDATE ewidencja_listow
SET status_doreczenia = 'W doręczeniu'
WHERE id_listu BETWEEN 23 AND 28;
```

B. Usuwanie zbędnych zasobów (DELETE): Procedura likwidacji środka trwałego, który uległ trwałemu zniszczeniu lub został wycofany z eksploatacji (np. kasacja starego roweru bez numeru rejestracyjnego).

SQL
```
DELETE FROM listonosze
WHERE imie = 'Janusz'
AND nazwisko = 'Przykładowy';
```